

how big hfts are build? like the HRT, or others teir 1 hfts
I want to know the initial stage as if I want to build my own
hft from scratch as Im developer so how to procedd and
what exactly to do and expect

Building something even remotely comparable to tier-1 firms like Hudson River Trading, Jane Street, or Citadel Securities is less about “writing a trading bot” and more about engineering an ultra-optimized distributed system tightly coupled with quantitative research.

I’ll break this into **how they are actually built**, then **what you should realistically do starting from scratch**.

1. What “big HFT” actually is (under the hood)

At tier-1 level, an HFT firm is composed of **four tightly integrated pillars**:

1) Market Data Infrastructure (the real edge starts here)

- Direct exchange feeds (ITCH, OUCH, FIX, binary protocols)
- FPGA / kernel bypass networking (Solarflare, DPDK)
- Co-location inside exchange data centers
- Custom feed handlers (C++)

👉 Goal: **nanosecond-level latency + deterministic parsing**

2) Strategy Engine (alpha generation)

- Microstructure models (order book imbalance, queue position)
- Statistical arbitrage / market making
- Signal pipelines (tick-by-tick)

👉 Written in:

- C++ (core execution)
- Python (research layer)

3) Execution System (OMS on steroids)

- Smart order routing
- Order lifecycle management (cancel/replace loops)
- Risk checks in microseconds

👉 This is where your OMS experience is relevant.

4) Risk + Monitoring

- Kill switches
 - Real-time PnL
 - Position limits
-

2. Reality check (important)

You **cannot replicate HRT-level HFT** as an individual because:

- No co-location → +100–500 μ s disadvantage
- No direct feeds → slower than SIP / broker APIs
- No exchange relationships
- No FPGA / custom NIC stack

👉 So forget *true HFT* initially.

Instead, aim for:

“Low-latency systematic trading” (retail-accessible HFT-lite)

3. What YOU should build (practical path)

Stage 1 — Build a “serious” trading system (not toy bots)

Start with:

A. Data layer

- Collect:
 - L2 order book

- Trades (tick data)
- Sources:
 - Binance, Bybit (crypto is best playground)
 - NSE is harder (no easy direct feeds)

👉 Store in:

- kdb+ (ideal but expensive)
- ClickHouse (good alternative)
- Parquet + S3 (cheap)

B. Event-driven architecture

Core pattern:

Market Data → Signal Engine → Execution Engine → Exchange



Use:

- Python (fast iteration)
- Async IO / Kafka / Redis streams

C. Build your first strategies

Focus on microstructure:

- Order book imbalance
- Spread capture
- Mean reversion (short horizon)
- Latency arbitrage (only within crypto)

Stage 2 — Build low-latency components

Now start getting serious:

Move critical path to C++

- Feed handler
- Order execution

Optimize:

- Lock-free queues
 - Memory pools
 - Zero-copy parsing
-

Stage 3 — Exchange connectivity

Instead of REST APIs:

- Use WebSocket for market data
- Use FIX / binary APIs if available

Crypto exchanges:

- Binance FIX
 - Deribit (good infra)
-

Stage 4 — Build your “mini OMS”

Since you already worked on OMS:

Add:

- Order state machine
 - Retry logic
 - Latency tracking
 - Risk checks
-

Stage 5 — Backtesting engine (most people skip this properly)

You need:

- Event-driven backtester (not candle-based)
 - Order book replay
 - Latency simulation
-

4. What your stack should look like

Initial stack (realistic for you)

- Python: research + orchestration
 - C++: execution path
 - Redis / Kafka: messaging
 - ClickHouse: storage
-

Advanced stack (closer to HFT)

- C++ everywhere
 - DPDK networking
 - Custom memory allocators
 - Kernel bypass
-

5. Timeline (realistic)

If you're serious:

0–3 months

- Build OMS + data ingestion
- Basic strategies

3–6 months

- Event-driven backtester
- Improve execution

6–12 months

- C++ migration
- Latency optimization

12+ months

- Possibly profitable system
-

6. Biggest mistakes people make

1. Starting with ML

- HFT $\neq$ deep learning first

2. Ignoring execution latency

3. Using candle data
4. Overfitting backtests
5. Thinking strategy > infrastructure

👉 In HFT:

Infrastructure quality = strategy viability

7. Where YOU have advantage

Given your background:

- OMS → execution layer advantage
- Python → fast research
- Crypto familiarity (likely) → easier access

👉 You are already ahead of most beginners.

8. If your goal is Tier-1 HFT job

Building this gives you:

- Strong system design story
- Low-latency experience
- Real trading exposure

Which is exactly what firms like:

- Hudson River Trading
- Tower Research Capital
- Jump Trading

look for.

9. Straight answer: what to do next

Do this in order:

1. Build crypto L2 data pipeline
2. Build event-driven strategy engine
3. Build OMS (you already can)

4. Deploy live with small capital
 5. Measure latency + slippage
 6. Rewrite hot path in C++
-

If you want, I can:

- Design a **complete system architecture for you (like HFT-lite blueprint)**
- Suggest **specific strategies that actually work in crypto microstructure**
- Or give a **project roadmap tailored to your current skillset + resume targeting HRT/Wolverine**

Just tell me how deep you want to go.



okay I already have very good experience in c++ also and good fast learner so I directly want to start with pure c++ and no python or other and also for other components directly wanna start with best possible options so now give guidelines agains in details and also give me example when HRT was founded and did they do the same so give me kind of journey of it so I can learn...

Good—this changes the plan materially. If you're committing to **pure C++ + performance-first from day 1**, you should structure this like a *mini prop firm*, not a "project".

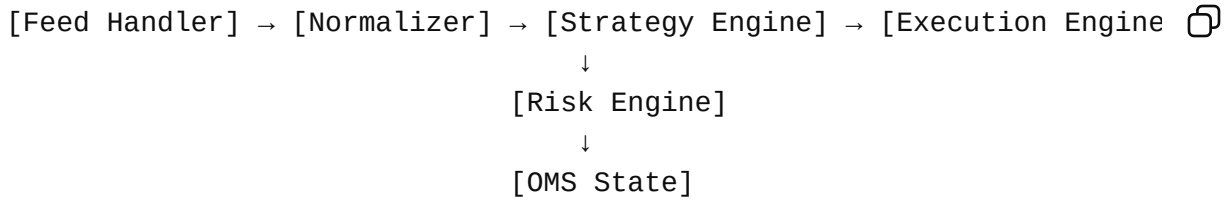
I'll give you:

1. **Exact system architecture (C++-first, production-grade)**
 2. **Build order (what to implement week-by-week)**
 3. **What tier-1 firms like Hudson River Trading actually did early on**
 4. **Where you should deviate (because you don't have their advantages)**
-

1. Your Target Architecture (C++-only, low-latency oriented)

Think in **process-level isolation**, not one monolith.

Core processes:



A. Feed Handler (Ultra-critical)

Goal: deterministic, zero-copy parsing

Requirements:

- Binary protocol parsing (no JSON)
- Lock-free ring buffer output
- Kernel bypass (later: DPDK)

Design:

</> C++

```

struct MarketUpdate {
    uint64_t ts_exchange;
    uint64_t ts_recv;
    uint32_t instrument_id;
    // price levels, etc
};
  
```

- Use:
 - mmap / preallocated buffers
 - SO_TIMESTAMPING (Linux)
 - busy polling (epoll is too slow for HFT tier)

B. Normalizer

- Converts raw feed → internal format
- Maintains order book (L2/L3)

👉 This is where you:

- Track queue position
 - Maintain price-time priority
-

C. Strategy Engine

Strict rules:

- **No heap allocation**
- **No locks**
- Pure function-like processing

Example:

</> C++



```
inline void on_tick(const MarketUpdate& u) {  
    auto imbalance = compute_imbalance(book);  
    if (imbalance > threshold) {  
        send_order(SIDE_BUY);  
    }  
}
```

D. Execution Engine

- Handles:
 - Order throttling
 - Replace/cancel loops
 - Exchange acks

👉 Needs:

- State machine per order
 - Microsecond-level timers
-

E. Risk Engine

Inline risk checks:

- Max position
- Max order size

- Kill switch

Must not block execution path

F. OMS State Store

- In-memory only (critical path)
 - Async persistence (disk)
-

2. Tech Stack (Best Possible from Start)

Since you want top-tier:

Language:

- C++20/23

Libraries:

- Networking: raw sockets → DPDK later
- Serialization: custom binary
- Logging: lock-free ring buffer logger

Data structures:

- Lock-free queues (SPSC)
 - Flat arrays over maps
 - Cache-line aligned structs
-

Hardware (don't ignore this)

Even early stage:

- CPU: high clock speed > cores
 - NIC: Intel X710 / Mellanox
 - Disable:
 - CPU frequency scaling
 - Hyperthreading (sometimes)
-

3. Build Order (THIS is where most fail)

Phase 1 (Weeks 1–3): Feed + Book

- Build feed handler (crypto first: Binance UDP/WebSocket hybrid)
- Build full L2 book
- Replay historical data

👉 Output:

- Deterministic book reconstruction
-

Phase 2 (Weeks 4–6): Strategy + Simulation

- Plug strategy into replay engine
- Add:
 - Fill simulation
 - Latency model

👉 This becomes your **research + backtesting system**

Phase 3 (Weeks 6–8): Execution Engine

- Build order state machine
 - Handle:
 - ACK / REJECT / FILL
 - Add retry logic
-

Phase 4 (Weeks 8–10): Live Trading (small capital)

- Connect to exchange
 - Run with minimal risk
-

Phase 5 (Months 3–6): Optimization

- Remove syscalls in hot path
 - NUMA pinning
 - Move to kernel bypass (DPDK)
-

4. What HRT Actually Did (Reality vs Myth)

Hudson River Trading was founded ~2002 by:

- Hudson River Trading founders

Early-stage reality:

They did NOT start with:

- FPGA
- Massive infra
- 100 engineers

They DID have:

1. Strong math + signal focus
2. Better infrastructure than competitors (even early)
3. Direct market access (huge advantage)

Their early system likely looked like:

- C++ core trading engine
- Simple stat arb / market making
- Direct exchange connectivity (key edge)

👉 Important:

Their "edge" was already institutional from day 1.

Evolution:

Stage 1:

- Simple strategies + decent infra

Stage 2:

- Scale infra (multi-market, multi-asset)

Stage 3:

- Invest heavily in latency (FPGA, colocation)

5. Critical Difference: You vs HRT

You do **NOT** have:

- Co-location
- Exchange relationships
- Proprietary data feeds

So if you copy them blindly → you fail.

6. What YOU should do differently

Focus on:

1. Crypto markets

- Less efficient
 - Retail-accessible
 - Real alpha exists
-

2. Strategy types that work without ultra-low latency

- Short-term mean reversion
 - Liquidity provision (wide spreads)
 - Cross-exchange arbitrage
-

3. Latency tier you should target:

Tier	Latency	Feasible?
HRT-level	<5 μ s	✗
Semi-HFT	50–200 μ s	⚠
Your target	500 μ s – 5 ms	✓

7. Non-negotiable engineering principles

If you want to be taken seriously:

1. Determinism

- Same input → same output ALWAYS

2. Zero GC / zero allocation in hot path

3. Full observability

- Latency per stage
 - Order lifecycle tracking
-

8. What to expect (brutally honest)

First 3 months:

- No profit
- Mostly infra bugs

6 months:

- Possibly breakeven

12 months:

- If done right → small edge
-

9. If your real goal = Tier-1 HFT job

This approach gives you:

- Real low-latency system experience
- C++ performance engineering
- Market microstructure understanding

Which is exactly what firms like:

- Hudson River Trading
- Jump Trading
- Tower Research Capital

care about.

10. Next step (if you want precision)

I can go one level deeper and give you:

- Exact **repo structure (like real HFT codebases)**
- A **minimal C++ code skeleton (feed → strategy → execution)**
- Or **specific strategies with math + implementation details**

Tell me which direction you want:

- “architecture deep dive”
- “code skeleton”
- “strategy alpha”



okay so now give me more detailed with developers perspective with system arch like these are all components this feed handler componet does this like stores market data (exactly specify what data like orderbook or tick data) and stores in redis or something else then this other component does this... and make the arch like what hrt or top tier hfts have

You're now asking the *right* question—this is where most people either build toy systems or something genuinely close to a prop-grade stack.

I'll lay this out like a **real HFT internal design doc**, from a developer's perspective, with:

- Clear component boundaries
- Exact data flowing between them
- Storage decisions (and where NOT to use Redis, etc.)
- What tier-1 firms (like Hudson River Trading) conceptually resemble—adapted to what you can actually build

0. First Principle: Two Separate Worlds

All serious HFT systems are split into:

1) Hot Path (latency critical)

- Runs entirely in memory
- No DB, no Redis, no syscalls (ideally)
- Microsecond constraints

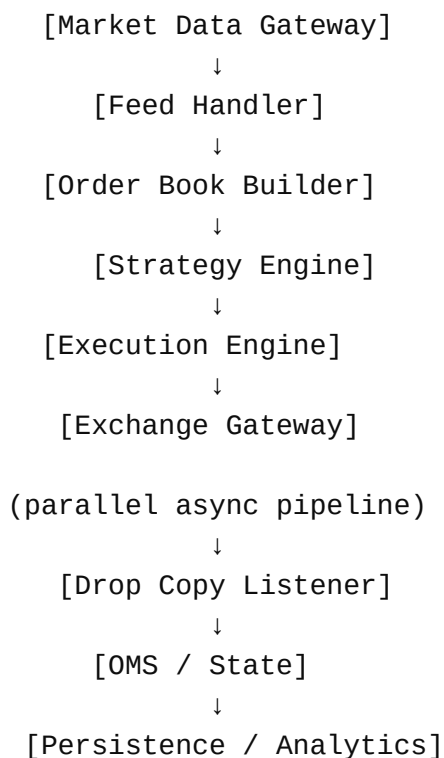
2) Cold Path (analytics / storage)

- Logging, research, monitoring
- Async, eventual consistency

👉 If you mix these → system is dead.

1. High-Level Process Architecture

Think **multi-process, shared-memory + lock-free queues**



2. Component-by-Component (Developer View)

2.1 Market Data Gateway

Responsibility:

- Network I/O only
- Reads raw packets from exchange

Input:

- UDP multicast / TCP / WebSocket (crypto)

Output:

- Raw binary messages

Key details:

- Use:
 - `recvmsg()` with batching
 - Busy polling
- Timestamp ASAP:

⌌ C++



```
ts_recv = rdtsc(); // or hardware timestamp
```

👉 NO parsing here

2.2 Feed Handler

Responsibility:

- Parse exchange protocol → internal struct

Input:

- Raw packets

Output:

⌌ C++



```
struct MarketEvent {  
    uint64_t ts_exchange;  
    uint64_t ts_recv;  
    uint32_t instrument_id;  
    EventType type; // trade / add / cancel / modify  
    double price;
```

```
double qty;  
uint64_t order_id;  
};
```

What data exactly:

- Order-level updates (L3 if possible)
- Trades (tick data)
- Book changes

👉 This is NOT “OHLC” or candles—**full microstructure data**

2.3 Order Book Builder

Responsibility:

Maintain real-time order book

Stores:

- Bid/Ask ladders
- Order queue (for L3)

Internal structures:

</> C++



```
struct PriceLevel {  
    double price;  
    double total_qty;  
    std::deque<Order> queue;  
};
```

Output:

</> C++



```
struct BookSnapshot {  
    double best_bid;  
    double best_ask;  
    double spread;  
    double imbalance;  
};
```

Critical:

- Deterministic replay
 - No dynamic allocation in hot path
-

2.4 Strategy Engine

Responsibility:

Generate signals + orders

Input:

- Book updates
- Trades

Output:

<> C++



```
struct OrderIntent {  
    Side side;  
    double price;  
    double qty;  
    StrategyId strategy_id;  
};
```

Example signals:

- Order book imbalance
- Microprice deviation
- Queue position advantage

👉 Runs in **single-threaded, pinned core**

2.5 Execution Engine

Responsibility:

Convert intent → actual orders

Handles:

- Order throttling
- Replace/cancel loops
- Exchange rate limits

Maintains:

</> C++



```
struct LiveOrder {  
    OrderId id;  
    Status status;  
    double remaining_qty;  
};
```

Output:

- Binary/FIX messages to exchange
-

2.6 Exchange Gateway

Responsibility:

- Protocol encoding/decoding
- Send/receive orders

Input:

- Order requests

Output:

- ACK / FILL / REJECT
-

2.7 Drop Copy Listener

Responsibility:

- Independent confirmation of fills
- Reconciliation

👉 Critical in real firms

2.8 OMS (State Manager)

Responsibility:

Maintain global trading state

Stores:

- Positions
- PnL
- Open orders

Important:

- NOT in hot path
 - Updated async
-

2.9 Persistence Layer (Cold Path)

What gets stored:

1. Market data

- Full order book events (compressed)
- Trades

2. Trading logs

- Orders sent
 - Fills received
 - Latency metrics
-

Storage choices:

Redis (NOT ideal for HFT core)

- Too slow for tick-level persistence
- Good only for:
 - Monitoring dashboards
 - Temporary state

Better:

- **ClickHouse** → analytics

- **Parquet + S3** → cheap storage
- **kdb+** → industry standard (if you had \$\$\$)

3. Data Flow (Precise)

Hot path (no storage):

```
NIC → Market Data Gateway
      → Feed Handler
      → Book Builder
      → Strategy
      → Execution
      → Exchange
```



Cold path:

```
Feed Handler → Logger → Disk
Execution → Logger → Disk
Drop Copy → OMS → DB
```



4. Inter-Process Communication

DO NOT use:

- REST
- gRPC
- Kafka (too slow for hot path)

Use:

- Lock-free ring buffers (SPSC)

Example:

```
<> C++

boost::lockfree::spsc_queue<MarketEvent>
```



Or custom ring buffer.

5. Threading Model

Ideal:

Component	Thread
Feed Handler	Core 1
Book Builder	Core 2
Strategy	Core 3
Execution	Core 4

👉 CPU pinning mandatory

6. Latency Budget (Realistic for you)

Stage	Target
Packet → parsed	< 5 μ s
Book update	< 5 μ s
Strategy	< 2 μ s
Execution send	< 5 μ s

Total:

👉 ~20–50 μ s internal

(External latency dominates anyway)

7. What Tier-1 Firms Add (Beyond This)

Firms like Hudson River Trading extend this with:

1. Hardware acceleration

- FPGA feed handlers

2. Custom networking

- Kernel bypass (DPDK, Solarflare OpenOnload)

3. Massive scale

- Thousands of strategies
- Multi-asset (equities, options, futures)

4. Advanced research infra

- Huge historical datasets
 - Simulation clusters
-

8. What You Should NOT Overbuild Initially

Avoid:

- FPGA
- Distributed clusters
- Multi-region systems

Focus on:

Single machine, perfectly optimized

9. Minimal Viable “Real HFT System”

If you build JUST this, you’re already ahead of 99%:

- C++ feed handler
 - Full L2 book
 - Event-driven strategy engine
 - Execution engine with order state machine
 - Tick-level logging
 - Replay backtester
-

10. If you want next level

I can now go deeper into:

Option A:

- Exact **C++ repo structure (folders, modules, build system)**

Option B:

- Full **code skeleton (interfaces + structs + flow)**

Option C:

- **Strategy design at microstructure level (what actually works)**

Option D:

- **Performance tuning (NUMA, cache lines, false sharing, etc.)**

Pick one, and I'll go very deep.



okay if I built this individually and can I earn profit from this?

Short answer: **yes, but not in the way most people imagine.**

You *can* become profitable with a solo, C++-first system—but not by competing with firms like Hudson River Trading or Jump Trading on raw latency. Your profitability will come from **choosing the right market + strategy + execution quality**, not from beating them at nanoseconds.

Let's break this down precisely.

1. Where profit actually comes from (for YOU)

Your edge will be:

1) Market inefficiency (not speed supremacy)

You target:

- Crypto (Binance, Bybit, Deribit)
- Smaller pairs / derivatives

- Regimes where liquidity is inconsistent

Not:

- US equities (dominated by firms like Citadel Securities)
-

2) Execution quality (your biggest advantage)

Most retail traders:

- Use REST APIs
- Have terrible order handling
- Don't manage queue position

You will:

- React faster (tens of μ s internally)
- Place better orders
- Cancel intelligently

👉 This alone creates measurable edge.

3) Microstructure awareness

Examples:

- Order book imbalance
- Microprice deviation
- Spread dynamics
- Liquidity shifts

This is where real alpha lives at your level.

2. Realistic profit expectations

Let's be very concrete.

Scenario A: Early stage (0–3 months)

- Likely outcome: **losses**
- Reasons:
 - Bugs
 - Bad fills

- Overfitting

Scenario B: Intermediate (3–6 months)

- Outcome:
 - Breakeven or small profit
- If:
 - Execution is solid
 - You control costs (fees/slippage)

Scenario C: Mature (6–12 months)

If done well:

- **1–5% monthly return** on deployed capital
- With scaling → meaningful income

Example:

Capital	2% / month
\$5k	\$100
\$20k	\$400
\$100k	\$2,000

👉 This is realistic. Not hype.

3. What determines if you succeed

1) Execution > Strategy (counterintuitive but true)

Two traders:

- Same strategy
- Different execution

The better execution wins.

2) Fees and rebates

In crypto:

- Maker rebates can be edge
- Taker fees kill you

👉 Many profitable systems are basically:

“execution + fee optimization”

3) Latency (but relative, not absolute)

You don't need:

- 1 μ s

You need:

- Faster than other API traders
-

4) Risk management

Without this:

- One bug = account blown
-

4. Where people fail (you should avoid)

1) Trying to compete with tier-1 HFTs

You lose instantly.

2) Ignoring slippage

Backtest $\neq$ reality

3) Overtrading

Fees eat profit

4) No replay system

If you can't replay → you can't improve

5. Strategies that actually work at your level

Focus here:

A. Passive market making (best starting point)

- Place bid/ask
 - Capture spread
 - Manage inventory
-

B. Micro mean reversion

- Short-term deviations
 - 1–10 second horizon
-

C. Cross-exchange arbitrage (still works in crypto)

- Price differences
 - Requires good infra
-

6. Your real edge (important)

Given:

- Strong C++
- OMS experience

You are in top ~5% of people attempting this.

👉 Most fail because:

- They are traders, not engineers

You're the opposite → advantage.

7. Brutal truth

You will NOT:

- Beat Hudson River Trading on latency
 - Make millions in 3 months
-

You CAN:

- Build a profitable system
 - Generate consistent returns
 - Build something that:
 - Gets you into HFT firms
 - Or scales into a small prop operation
-

8. Best strategy for YOU (specific advice)

Do this:

Step 1:

- Crypto perpetual futures (Binance)

Step 2:

- Build market making system

Step 3:

- Focus on:
 - Spread capture
 - Inventory control
 - Latency measurement

Step 4:

- Scale capital slowly
-

9. Expected journey

Month 1–2:

- System building
- No profit

Month 3–4:

- First working strategy

Month 6:

- Stable system

Month 12:

- Real edge (if disciplined)

10. Final answer (no sugarcoating)

Yes—you can make money.

But:

This is a **systems engineering problem first, trading problem second.**

If you build:

- Correct architecture
- Deterministic system
- Good execution

Then profitability is **very realistic.**

If you want next step, I'd suggest:

👉 I can design:

- A **market making strategy with exact formulas + C++ logic**
- Or a **PnL + fill simulation model (critical for profitability)**

Tell me which one you want.



I can design:

A market making strategy with exact formulas + C++ logic
Or a PnL + fill simulation model (critical for profitability)

do both for me

Good—this is exactly the layer where systems turn into actual money.

I'll give you both:

1. A production-grade market making model (math + logic)
2. A realistic PnL + fill simulator (what most people get wrong)
3. C++-style structures and flow you can directly plug into your engine

No fluff—this is how you'd actually implement it.

1. Market Making Strategy (Microstructure-Driven)

You're not just quoting bid/ask—you're pricing **risk + inventory + short-term alpha**.

1.1 Core Idea

You compute a **fair price**, then skew your quotes around it.

Fair price (microprice):

$$P_{micro} = \frac{P_{ask} \cdot Q_{bid} + P_{bid} \cdot Q_{ask}}{Q_{bid} + Q_{ask}}$$

Where:

- P_{bid}, P_{ask} : best prices
- Q_{bid}, Q_{ask} : sizes at best levels

👉 This gives **directional bias from order book pressure**

1.2 Inventory Skew (THIS is critical)

You adjust quotes based on your position:

$$P_{skew} = P_{micro} - \alpha \cdot Inventory$$

- If long → lower your quotes (encourage selling)
- If short → raise quotes

1.3 Spread Model

Your spread should adapt:

$$Spread = base + \beta \cdot Volatility + \gamma \cdot Inventory^2$$

1.4 Final Quotes

$$P_{bid}^{quote} = P_{skew} - \frac{Spread}{2}, \quad P_{ask}^{quote} = P_{skew} + \frac{Spread}{2}$$

1.5 When NOT to quote

Do NOT place orders if:

- Spread too tight
- Volatility spike
- Inventory too large
- Latency spike detected

2. C++ Strategy Logic (Plug-and-Play Style)

State:

 C++



```
struct StrategyState {
    double inventory;
    double cash;
    double last_mid;
```

```
double volatility;  
};
```

Core function:

</> C++



```
inline void on_book_update(const Book& book, StrategyState& s) {  
    double best_bid = book.best_bid;  
    double best_ask = book.best_ask;  
    double bid_qty = book.bid_qty;  
    double ask_qty = book.ask_qty;  
  
    double microprice =  
        (best_ask * bid_qty + best_bid * ask_qty) /  
        (bid_qty + ask_qty);  
  
    double skew = microprice - alpha * s.inventory;  
  
    double spread = base_spread  
        + beta * s.volatility  
        + gamma * s.inventory * s.inventory;  
  
    double bid_quote = skew - spread * 0.5;  
    double ask_quote = skew + spread * 0.5;  
  
    if (should_quote(book, s)) {  
        place_or_update_orders(bid_quote, ask_quote);  
    } else {  
        cancel_all();  
    }  
}
```

2.1 Order Placement Logic

</> C++



```
void place_or_update_orders(double bid, double ask) {  
    if (!has_bid_order())  
        send_limit_order(BUY, bid);  
    else  
        modify_order(bid_order_id, bid);  
}
```

```
if (!has_ask_order())
    send_limit_order(SELL, ask);
else
    modify_order(ask_order_id, ask);
}
```

3. Fill Model (Most Important Part)

If your fill model is wrong → your strategy is fake.

3.1 Queue Position Model

You must track:

</> C++



```
struct QueuePosition {
    double ahead_qty;
    double my_qty;
};
```

3.2 Fill Condition

You get filled when:

Traded Volume $\geq$ *Ahead Quantity*

3.3 Partial Fill

</> C++



```
double fill_qty = min(trade_volume - ahead_qty, my_qty);
```

3.4 Queue Update

Every book event:

- Add orders ahead of you
- Remove when trades happen

4. PnL Model (Realistic)

4.1 Realized PnL

$$PnL_{realized} = \sum (Sell\ Price - Buy\ Price) \cdot Quantity$$

4.2 Mark-to-Market PnL

$$PnL_{mtm} = Inventory \cdot (MidPrice - AvgEntryPrice)$$

4.3 Fees (critical)

$$PnL_{net} = PnL_{realized} + PnL_{mtm} - Fees$$

4.4 Adverse Selection (hidden killer)

You must track:

- Fill → price moves against you → loss

Metric:

</> C++



```
double adverse_move = mid_price_after_fill - fill_price;
```

5. C++ PnL Engine

</> C++



```

struct PnLState {
    double realized = 0;
    double inventory = 0;
    double avg_price = 0;
};

void on_fill(PnLState& p, Side side, double price, double qty) {
    if (side == BUY) {
        double new_inventory = p.inventory + qty;
        p.avg_price = (p.avg_price * p.inventory + price * qty) / new_inve
        p.inventory = new_inventory;
    } else {
        p.realized += (price - p.avg_price) * qty;
        p.inventory -= qty;
    }
}

```

6. Backtesting Engine (Event-Driven)

You must replay:

- Order book updates
- Trades
- Your orders

Loop:

</> C++



```

for (auto& event : events) {
    book.apply(event);
    strategy.on_book_update(book);
    simulator.match_orders(event);
}

```

7. Reality Enhancements (DO NOT SKIP)

Add:

1. Latency simulation

- Order delay (e.g., 1–5 ms)

2. Cancel delay

- You don't cancel instantly

3. Partial fills

- Always partial, rarely full
-

8. Why this works

This strategy makes money from:

- Spread capture
- Inventory mean reversion
- Microstructure inefficiencies

NOT prediction.

9. What will break first

Expect issues with:

1. Fill model too optimistic
 2. Overtrading → fees kill profit
 3. Inventory drift
 4. Latency mismatch
-

10. What to do next

Implement in this exact order:

1. Order book + replay
 2. Fill simulator (queue-based)
 3. PnL engine
 4. Strategy logic
 5. Live execution
-

If you want, next I can:

- Turn this into a **complete C++ repo structure (production-style)**

- Or give a **more advanced model (Avellaneda–Stoikov style with stochastic control)**
- Or help you **tune parameters (alpha, beta, gamma) based on real data**

Tell me the next step.

